Debugging the Removals System

Mindaugas Taujanskas

I. Registering service-provider causes UserAlreadyExistsError

Upon attempting to register a company, an initial error is raised:

```
psycopg2.errors.CheckViolation: new row for relation
     "businesses" violates check constraint "
    chk_businesses__utr_no"
DETAIL:  Failing row contains (1, example business
    name, asdasdas, , , 123).
```

This stems from the following Python code which calls to register a business for a registered user:

```
user = self.register_user_from_detail_form(
  service_provider_details,
  "service-provider"
)
business = Business.create_for_user(
  user.token,
  business_name=form_data['business-name'],
  crn_no=form_data['crn'],
  vat_no=form_data['vat-number'],
  utr_no=form_data['utr-number'],
  num_employees=form_data['nr-employees']
)
```

It should be noted that the VAT/UTR fields are specified to be optional by the business requirements, however the relevant SQL check-constraint `chk_businesses__utr_no` (and also the VAT) are implemented as such:

```
CREATE TABLE Businesses (
  ...
  vat_no          TEXT UNIQUE,
  utr_no          TEXT UNIQUE,
  ...
  CONSTRAINT CHK_Businesses__vat_length
  CHECK (LENGTH(vat_no) = 11),
  CONSTRAINT CHK_Businesses__utr_no
  CHECK (LENGTH(utr_no) = 10)
);
```

This is contrary to the business requirements, and thus causes the Python code to fail in two ways. Firstly, the fields are notated `UNIQUE`, and so they cannot be `NULL`, as more than one duplicate value would immediately fail. Secondly, the check-constraint doesn't account for the fact either field could be empty. This can be ameliorated with the following changes:

```
CREATE TABLE Businesses (
  ...
  vat_no          TEXT,
  utr_no          TEXT,
  ...
  CONSTRAINT CHK_Businesses__vat_length
  CHECK (COALESCE(LENGTH(vat_no) = 11, TRUE)),
  CONSTRAINT CHK_Businesses__utr_no
  CHECK (COALESCE(LENGTH(utr_no) = 10, TRUE))
);
```

This removes the uniqueness constraint, and allows the field to either be empty, or text with length 11/10 respectively. The `UNIQUE` constraint can be enforced at trigger-level on pre-insert onto the businesses table.

The second error is caused when the user attempts to register again with the same details:

```
File "controllers\role_selection.py", line 105, in
    register_user_from_detail_form
  user = register_user(**{
    "forename": self.user_auth['forename'],
    ...<4 lines>...
    "role": role
  })
File "models\user.py", line 71, in register_user
  data = db_errors.unwrap_result(db.
      proc_register_user(**details))
File "models\db_errors.py", line 99, in
    unwrap_result
  raise_from_error_t(result.error)
  ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^
File "models\db_errors.py", line 77, in
    raise_from_error_t
  raise exc_cls(f"Result returned error: {error.
      message!r}")
removals_system.exceptions.auth_exceptions.
    UserAlreadyExistsError: Result returned error: '
    Email already exists'
```

This follows from the code given before:

```
user = self.register_user_from_detail_form(
  service_provider_details,
  "service-provider"
)
business = Business.create_for_user(
  user.token,
  business_name=form_data['business-name'],
  crn_no=form_data['crn'],
  vat_no=form_data['vat-number'],
  utr_no=form_data['utr-number'],
  num_employees=form_data['nr-employees']
)
```

The registration always succeeds, but the business creation doesn't, so then if the user attempts to move forward again after fixing their details, it will attempt to recreate the account. This can be solved in a couple of ways:

1) Roll-back the user creation if the business creation fails
2) Ensure the database logic is correct as to prevent this edge-case
3) Persist the user-token so that registration is not repeated in the same form

The simplest method we will apply is (2), however note that this fall shorts of addressing the real issue of weak state management. Also note that the `Business.create_for_user` call should be rewritten to:

```
business = Business.create_for_user(
  user.token,
  business_name=form_data['business-name'],
  crn_no=form_data['crn'],
  vat_no=form_data['vat-number'] or None,
  utr_no=form_data['utr-number'] or None,
  num_employees=form_data['nr-employees']
)
```

As an empty string is not equivalent to `NULL`. After applying these changes, the code works as intended.

II. Creating a Business as a Customer

During the creation of tests for the business creation logic, a bug was discovered regarding the permissibility of customers creating business resources, contrary to the business logic that indicates only service providers may register businesses. The test case is as follows:

```
def test_appropriate_role_permission(
  db_guest_cursor,
  with_valid_user
) -> None:
  *_, session = with_valid_user("customer")
  fetch_result = db.proc_create_business(
    session.data.token,
    business_name="Sample business name",
    vat_no=gen_rand_alnum(length=11),
    crn_no=gen_rand_alnum(length=8),
    utr_no=gen_rand_alnum(length=10),
    num_employees=2
  )
  assert not fetch_result.success
```

The case fails on `assert not fetch_result.success`, indicating that the database registered the business resource despite the registering user being a customer. To ensure the database handles this validation, check the procedural definition for the corresponding function:

```
CREATE FUNCTION create_business(
  p_token TEXT,
  ...
)
RETURNS result_t AS $$
DECLARE
  user_session JSON;
  stored_business_id INTEGER;
BEGIN
  user_session := decode_token(p_token);

  IF user_session IS NULL THEN
    RETURN make_error_result(
      'INVALID_SESSION',
      'Invalid session token'
    );
  ELSIF user_session->>'user_role' <> 'service-
      provider' THEN
    RETURN make_error_result(
      'INSUFFICIENT_PERMISSIONS',
      'Invalid role for creating business'
    );
  END IF;

  ...

  RETURN make_success_result();
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

As read, this appears to appropriately check the session data for the user role and is thus non-problematic, but upon further evaluation `user_session->>'user_role'` contains a subtle typo, and must be `user_session->>'role'`, as defined by the registration procedures:

```
token := sign(
  json_build_object(
    'user_id', new_user_id,
    'email', p_email,
    'role', p_user_role  -- <<< HERE
  ),
  utils.get_app_config_value('jwt_secret'),
  'HS256'
);
```

After correcting the typo, the test passes, meaning our business logic is correctly implemented at the database-level.